



Week 11 -Tree

☰ Tags	active
🔗 Cover	
☰ Notebook	NB3: Tree & Graph
🔗 URL	https://www.techinterviewhandbook.org/algorithms/tree/
☰ type	Tree



Tree — The Hierarchical Data Structure



Quick Idea

A **tree** is a way to represent data in a **hierarchy** — like a **family tree** or a **folder system**. Contains **nodes** connected by **edges**



Core Rules

- **Root Node, Parent → Child** 👤 – Every node (except root) has *exactly one parent*. Each node can be connected to many children
- **No Cycles** ↻🚫 – You can't loop back up; it's *acyclic* (*không có chu trình*)



Binary Tree (for Interviews!)

- **Each node has ≤ 2 children**

▼ A **Binary Search Tree (BST)** is a special one where: `Left < Root < Right`



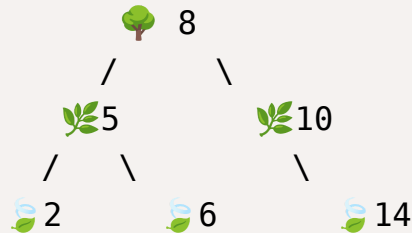
Binary Search Tree (BST)



Rule

- All nodes in the **left** are **smaller** 🌿
- All nodes in the **right** are **larger** 🌿

🌿 Visualize



➡ Explanation:

- 5 and 2,6 are smaller than 8 → go left ✓
- 10 and 14 are larger than 8 → go right ✓

Common terms

https://truongdinhtri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/Tree-Terms-Visualize.html



Binary Tree

- Binary means 2, so nodes in a binary tree have a maximum of 2 children

1. Binary tree terms

- Complete binary tree, full binary trees @ [Heap Sort](#) ($O(\log(n))$).

▼ A balanced binary tree

https://truongdinhdttri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/%F0%9F%8C%B3%20Complete%20Balanced%20Tree%20Visualizer.html

2. Traversals

WILL PUT HTML FILES TO DEPLOY TO VERCEL

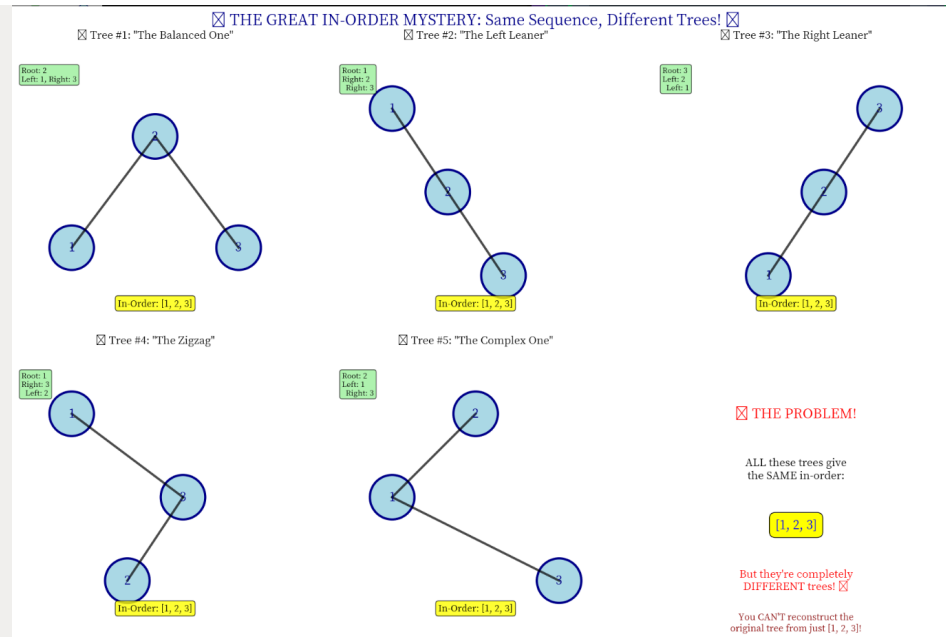


https://truongdinhtri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/Tree%20Traversal%20Masterclass%20-%20Enhanced%20Edition.html



https://truongdinhtri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/%F0%9F%8C%B3%20Ultra-Compact%20Tree%20Visualizer.html

▼ **Note that in-order traversal of a binary tree is insufficient to uniquely serialize a tree. Pre-order or post-order traversal is also required.**



THE SOLUTION: Combination Power!

You need **In-Order + (Pre-Order OR Post-Order)** to uniquely serialize!

In-Order + Pre-Order

Tree #1 (Balanced):
In-Order: [1, 2, 3]
Pre-Order: [2, 1, 3]

Tree #2 (Right Leaner):
In-Order: [1, 2, 3]
Pre-Order: [1, 2, 3]

✓ Now they're DIFFERENT!

Why it works: Pre-order tells us ROOT positions!

In-Order + Post-Order

Tree #1 (Balanced):
In-Order: [1, 2, 3]
Post-Order: [1, 3, 2]

Tree #2 (Right Leaner):
In-Order: [1, 2, 3]
Post-Order: [3, 2, 1]

✓ Now they're DIFFERENT!

Why it works: Post-order tells us LEAF-to-ROOT order!

KEY TAKEAWAYS

🚫 In-Order Alone =
AMBIGUOUS

Multiple trees can produce the same in-order sequence

✓ In-Order + Pre-Order =
UNIQUE

This combination uniquely identifies any tree

✓ In-Order + Post-Order =
UNIQUE

This combination also uniquely identifies any tree

🧠 Why This Matters

For tree serialization, data storage, and network transmission!

Binary search tree (BST)

- In-order traversal of a BST will give you **all elements in sorted order**.
- Be very familiar **with the properties of a BST and validating that a binary tree is a BST**. This comes up more often than expected.

- When a question involves a BST, the interviewer is **usually looking for a solution which runs faster than $O(n)$** .

▼ More explain



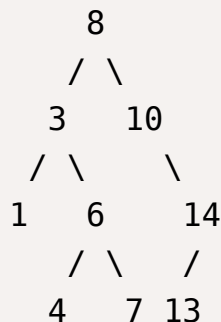
1. What's a Binary Search Tree (BST)?

A **BST** is a special binary tree that follows **3 golden rules**:

Rule	Description
♦ Left Subtree	Contains only nodes with values smaller than the parent
♦ Right Subtree	Contains only nodes with values greater than the parent
♦ Applies Globally	The rule must hold for all nodes , not just direct children



Visualization



✅ This is a **valid BST**, because:

- Left of 8 $\rightarrow \{1, 3, 4, 6, 7\} < 8$
- Right of 8 $\rightarrow \{10, 13, 14\} > 8$



2. What is In-order Traversal?

"In-order" means we visit the tree like this:

| Left $\rightarrow$ Root $\rightarrow$ Right

Example on that BST:

- 1 Go to the **left** subtree
- 2 Visit the **current** node
- 3 Go to the **right** subtree

🧩 **Step-by-step result:**

1 → 3 → 4 → 6 → 7 → 8 → 10 → 13 → 14

✨ **Sorted order!**

That's why doing in-order traversal on a BST gives a sorted list.

✅ 3. Validating a BST



https://truongdinhdttri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/BST%20Visualizer%20-%20Full%20View%20Edition.html

To check if a tree is a valid BST, you ensure that **every node** follows the BST rule with respect to **all its ancestors**.

🧠 **Intuition:**

Each node must be:

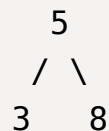
- **Greater than** everything in its left side
- **Smaller than** everything in its right side

🧩 **Python Code**

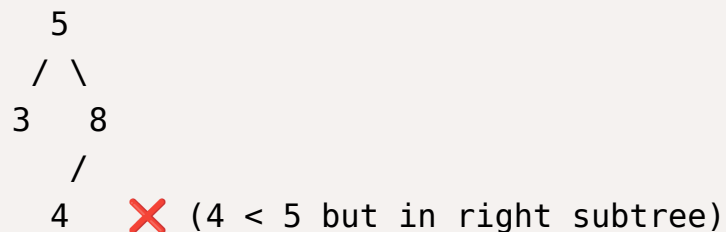
```
def isValidBST(root, min_val=float('-inf'), max_val=float('inf')):
    if not root:
        return True
    if not (min_val < root.val < max_val):
        return False
    return (
        isValidBST(root.left, min_val, root.val) and
        isValidBST(root.right, root.val, max_val)
    )
```

Example

Valid BST



Invalid BST



4. "Faster than O(n)" — Why They Say That

👉 In a **normal list**, finding a value = $O(n)$ (you scan everything).

👉 In a **BST**, you can skip half the tree each step — like binary search.

🔍 Search Algorithm

If the target is smaller → go left.

If larger → go right.

🕒 Time complexity

- **Balanced BST:** $O(\log n)$ ✓
- **Unbalanced (like a chain):** $O(n)$ ✗

🦆 Python Example

```
def searchBST(root, target):
    if not root:
        return False
    if root.val == target:
        return True
    elif target < root.val:
        return searchBST(root.left, target)
    else:
        return searchBST(root.right, target)
```

🔮 ✨ Summary Table

Concept	Key Idea	Example / Visualization
🌳 BST	Left < Node < Right	3 <- 5 -> 8
🔄 In-order	Left → Node → Right	Gives [1, 3, 4, 6, 7, 8, 10, 13, 14]
✓ Validation	Check all nodes follow BST rule globally	4 can't be in right subtree of 5

Concept	Key Idea	Example / Visualization
⚡ $O(\log n)$	Use BST structure for faster search	Skip half each step

Time complexity

Operation	Big-O
Access	$O(\log(n))$
Search	$O(\log(n))$
Insert	$O(\log(n))$
Remove	$O(\log(n))$

Space complexity of traversing balanced trees is $O(h)$ where h is the height of the tree, while traversing very skewed trees (which is essentially a linked list) will be $O(n)$.

Things to look out for during interviews

▼ You should be very familiar with writing pre-order, in-order, and post-order traversal recursively. As an extension, challenge yourself by writing them iteratively. Sometimes interviewers ask candidates for the iterative approach, especially if the candidate finishes writing the recursive approach too quickly.

What Your Teacher Really Meant

When you're in an interview and you get a **binary tree traversal** question — the interviewer wants to see:

1. 🧠 **Understanding:** Do you *really* know how a traversal works — not just memorized it?

2.  **Problem-solving skill:** Can you handle both recursion and iteration (real-world devs can't always rely on recursion)?
3.  **Depth of mastery:** If you finish the recursive one too fast, they'll say "Nice! Can you do it iteratively?" — that's their way of testing your deeper logic muscles.

So basically:

 "If you can write recursive easily — good. But if you can also simulate recursion manually using a stack — you're elite."

Recursion vs Iteration (The Core Idea)

Concept	Recursive Traversal	Iterative Traversal
Who manages the call stack?	Python does it for you automatically	You do it manually using your own stack (a list)
Feels like:	Elegant, short, easy to read	Logical, step-by-step control
Used in:	Simple problems	Performance-sensitive, memory-controlled problems
Why interviews love it:	Tests understanding	Tests mastery

Visualization Tip

Imagine recursion like a **magic butler** who remembers where you came from. Iteration means you **become your own butler**, carrying a notebook (stack ) and noting every room (node) you've visited.

Let's Code Beautifully Side-by-Side

We'll use a simple binary tree:

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

🌸 1. Pre-order Traversal (Root → Left → Right)

👉 Visit the node *before* its children.

Recursive

```
def preorder_recursive(root):
    if not root:
        return
    print(root.val, end=' ')
    preorder_recursive(root.left)
    preorder_recursive(root.right)
```

Iterative

```
def preorder_iterative(root):
    if not root:
        return
    stack = [root]
    while stack:
        node = stack.pop()
        print(node.val, end=' ')
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
```

🧩 **Why:** Stack reverses the normal order, so push `right` before `left`.

🌸 2. In-order Traversal (Left → Root → Right)



https://truongdinhdttri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/Inorder-Traversal-Interactive-Tool.html

👉 Visit left subtree first, then the root, then right.

Recursive

```
def inorder_recursive(root):
    if not root:
        return
    inorder_recursive(root.left)
    print(root.val, end=' ')
    inorder_recursive(root.right)
```

🧠 Logic:

You “walk down left” until there’s no more, then backtrack.

Iterative

```
def inorder_iterative(root):
    stack = []
    current = root
    while stack or current:
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        print(current.val, end=' ')
        current = current.right
```

🌻 3. Post-order Traversal (Left → Right → Root)



https://truongdinhtri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%200Deployed/From%20Preorder%20to%20Postorder%20-%20Interactive%20Discovery%20Tool.html

👉 Visit node *after* its children.

Recursive

```
def postorder_recursive(root):
    if not root:
        return
    postorder_recursive(root.left)
    postorder_recursive(root.right)
    print(root.val, end=' ')
```

Iterative (using two stacks)

```
def postorder_iterative(root):
    if not root:
        return
    stack1, stack2 = [root], []
    while stack1:
        node = stack1.pop()
        stack2.append(node)
        if node.left:
            stack1.append(node.left)
```

```

    if node.right:
        stack1.append(node.right)
    while stack2:
        print(stack2.pop().val, end=' ')

```

🎯 **Alternative:** one-stack version exists, but two-stacks version is clearer for interviews.

The Big Picture to Remember

Traversal	Order	Recursion Mindset	Iteration Trick
Pre-order	Root → L → R	Visit first	Push right first
In-order	L → Root → R	Go left until null	Use current + stack
Post-order	L → R → Root	Visit last	Two stacks or peek technique

Interview Tip

If you're ever asked:

| "Can you write the iterative version?"

You can say:

| "Sure! Let me simulate the recursion manually with a stack."

Then start confidently — because now you understand not just the *code*, but the *reasoning* behind it. 🌟

Corner cases

- Empty tree
- Single node

- Two nodes
- Very skewed tree (like a linked list)

Common routines



https://truongdinhtri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/Tree-routines.html

▼ Build a tree from an array

```
def build_tree(arr, i=0):  
    if i >= len(arr) or arr[i] is None:  
        return None  
    node = TreeNode(arr[i])  
    node.left = build_tree(arr, 2*i + 1)  
    node.right = build_tree(arr, 2*i + 2)  
    return node
```

[1,2,3,4,5,6,7]

- Insert value
- Delete value
- Count number of nodes in tree
- Whether a value is in the tree

- Calculate height of the tree
- Binary search tree
 - Determine if it is a binary search tree
 - Get maximum value
 - Get minimum value

▼ Clean, copy-paste Python 3 implementation

```
# bst_full.py
from typing import Optional, Tuple

class TreeNode:
    def __init__(self, val: int):
        self.val = val
        self.left: Optional['TreeNode'] = None
        self.right: Optional['TreeNode'] = None
    def __repr__(self):
        return f"TreeNode({self.val})"

class BinarySearchTree:
    def __init__(self):
        self.root: Optional[TreeNode] = None

    # Insert (duplicates go to the right by policy)
    def insert(self, val: int) -> None:
        def _insert(node: Optional[TreeNode], v: int) -> T
reeNode:
            if node is None:
                return TreeNode(v)
            if v < node.val:
                node.left = _insert(node.left, v)
            else:
                node.right = _insert(node.right, v)
            return node
```

```

        self.root = _insert(self.root, val)

# Delete
def delete(self, val: int) -> None:
    def _min_node(node: TreeNode) -> TreeNode:
        while node.left:
            node = node.left
        return node

    def _delete(node: Optional[TreeNode], v: int) -> Optional[TreeNode]:
        if node is None:
            return None
        if v < node.val:
            node.left = _delete(node.left, v)
        elif v > node.val:
            node.right = _delete(node.right, v)
        else:
            # Case: one or zero children
            if node.left is None:
                return node.right
            if node.right is None:
                return node.left
            # Case: two children -> use inorder successor
            succ = _min_node(node.right)
            node.val = succ.val
            node.right = _delete(node.right, succ.val)
        return node

    self.root = _delete(self.root, val)

# Count nodes
def count_nodes(self) -> int:
    def _count(n: Optional[TreeNode]) -> int:
        if n is None:

```

```

        return 0
    return 1 + _count(n.left) + _count(n.right)
return _count(self.root)

# Contains (iterative)
def contains(self, val: int) -> bool:
    node = self.root
    while node:
        if val == node.val:
            return True
        node = node.left if val < node.val else node.r
ight
    return False

# Height (number of nodes on longest path from root to
leaf)
def height(self) -> int:
    def _h(n: Optional[TreeNode]) -> int:
        if n is None:
            return 0
        return 1 + max(_h(n.left), _h(n.right))
    return _h(self.root)

# Is BST? using bounds
def is_bst(self) -> bool:
    def _is_bst(n: Optional[TreeNode], lo: float, hi:
float) -> bool:
        if n is None:
            return True
        if not (lo < n.val < hi):
            return False
        return _is_bst(n.left, lo, n.val) and _is_bst
(n.right, n.val, hi)
    return _is_bst(self.root, float('-inf'), float('in
f'))

```

```

# Get min / max
def get_min(self) -> Optional[int]:
    node = self.root
    if node is None:
        return None
    while node.left:
        node = node.left
    return node.val

def get_max(self) -> Optional[int]:
    node = self.root
    if node is None:
        return None
    while node.right:
        node = node.right
    return node.val

# Small utility to print in-order (sorted list)
def inorder_list(self) -> list:
    out = []
    def _inorder(n: Optional[TreeNode]):
        if n is None:
            return
        _inorder(n.left)
        out.append(n.val)
        _inorder(n.right)
    _inorder(self.root)
    return out

# Quick demonstration
if __name__ == "__main__":
    bst = BinarySearchTree()
    for v in [5,3,8,1,4,7,9]:
        bst.insert(v)
    print("In-order:", bst.inorder_list())
    print("Contains 4:", bst.contains(4))

```

```
print("Count:", bst.count_nodes())
print("Height:", bst.height())
print("Is BST:", bst.is_bst())
print("Min / Max:", bst.get_min(), bst.get_max())
bst.delete(8)
print("After delete 8:", bst.inorder_list())
```

Technique

1. Use recursion

▼ Recursion is the most common approach for traversing trees. **When you notice that the subtree problem can be used to solve the entire problem, try using recursion.**



Why recursion works so perfectly with trees

Think of a **tree** like a collection of *mini-trees* (subtrees).

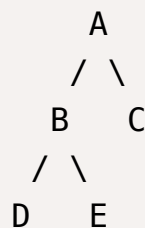
Each node is the **root** of its own smaller tree!

That's why recursion feels *natural* here —

because every subtree is a **smaller version of the same problem** 💡



Visualization: Tree = Node + Subtrees



To solve something for the whole tree (**A**),
you can **let recursion handle** the smaller trees (**B** and **C**).

You just:

1. Do something on the current node (e.g., print it 🎯)
2. Let recursion handle the left subtree 🌿
3. Let recursion handle the right subtree 🌿

That's literally **tree recursion** in one sentence! 💥



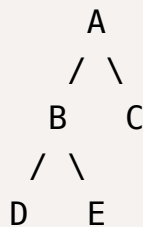
Example: Print all nodes in Pre-order

| Pre-order means: Root → Left → Right

```
def preorder(node):  
    if not node: # 🧱 base case  
        return  
    print(node.val) # 🎯 visit current node  
    preorder(node.left) # 🌿 go left  
    preorder(node.right) # 🌿 go right
```



Visualization



Steps:

1. Visit A ✅

2. Go left → visit B ✓
3. Go left again → visit D ✓
4. Backtrack (D done), go right → E ✓
5. Backtrack (B done), go right → C ✓

Output: `A B D E C`

💡 Each recursive call is like saying:

| “You handle your own subtree — I trust you.” 😊

▼ When using recursion, **always remember to check for the base case, usually where the node is `null`**.

⚙️ The Base Case (🚪 the exit door)

The **base case** is what *stops* recursion —
otherwise, your function will keep calling itself forever 🌀

Usually:

```
if node is None:  
    return
```

That means:

When there's no node (we've hit the bottom of the tree) → stop!

▼ Sometimes it is possible that your recursive function needs to return two values.

📁 When recursion returns two values

Sometimes you don't just traverse —
you need to **collect two different pieces of info** from each subtree.

Example: “Find the height of tree and whether it's balanced”

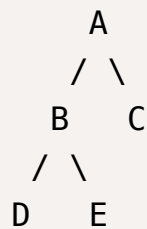
```
def check_balanced(node):
    if not node:
        return (0, True) # height = 0, balanced = True

    left_height, left_balanced = check_balanced(node.left)
    right_height, right_balanced = check_balanced(node.right)

    height = 1 + max(left_height, right_height)
    balanced = (abs(left_height - right_height) <= 1
                and left_balanced and right_balanced)

    return (height, balanced)
```

Visualization



Each subtree call returns:

- Its **height**
- Whether it's **balanced**

Then the parent (A) combines those results!

💬 So recursion here is like:

"Hey B, give me your height & balance."

"Hey C, give me yours too."

“Okay, I’ll combine your answers and return mine upward.” ↑

“Recursion in trees works because each subtree is just a smaller version of the same tree.” 🌳

So, when you solve a problem for one node + use recursion for subtrees, you’re actually solving the entire tree effortlessly!

2. Traversing by level

When you are asked to traverse a tree by level, use

breadth-first search

▼ More explain

🌈 Level Order Traversal (Breadth-First Search – BFS)

🧠 Intuition (core idea)

When someone says:

“Traverse the tree by level”

They mean:

👉 Visit all nodes in the **same depth** before moving deeper.

So instead of diving deep (like recursion does),

you go **wide**, level by level — from top to bottom 📅

🌳 Visualization

Example tree:

```
  A
 / \
```



BFS visits in this order:

👉 A

👉 B, C

👉 D, E, F

Output:

A → B → C → D → E → F

🚗 How it works (the queue magic 🎩)

Since recursion naturally goes deep (DFS),

we use a queue (FIFO structure) to control the order manually.

💡 Idea:

1. Start with the **root** in the queue.
2. Pop one node → visit it.
3. Push its **children** into the queue.
4. Repeat until the queue is empty.

This way, children always wait their turn after their parent — giving us a *perfect level-by-level order!* ✅

🧩 Python Code Example

```
from collections import deque

def level_order(root):
    if not root:
```

```
        return []

    queue = deque([root])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node.val)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    return result
```

Visualization Step-by-Step

Queue: [A]
→ Visit A 
Enqueue B, C
Queue: [B, C]

→ Visit B 
Enqueue D, E
Queue: [C, D, E]

→ Visit C 
Enqueue F
Queue: [D, E, F]

→ Visit D 
→ Visit E 

→ Visit F 
Queue: []

 Result: [A, B, C, D, E, F]

BFS by Level (grouped output)

Sometimes interviewers want *each level separated* — like this:

```
[  
  [A],  
  [B, C],  
  [D, E, F]  
]
```

When to use BFS

Situation	Why BFS fits
Printing tree level by level	Visits layer by layer
Finding shortest path in unweighted tree	Explores nearest first
Problems like "right side view", "average per level"	Each level is processed together

3. Summation of nodes

If the question involves summation of nodes along the way, be sure to check whether nodes can be negative.

▼ More explain for summation of nodes

Summation of Nodes in a Tree

When a problem says something like:

"Find the path with the maximum sum"

or

"Return the sum of all nodes in the tree"

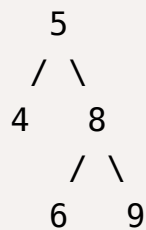
It's about combining values along paths 🌿

But — here comes the **tricky part** 📌

⚠️ The warning: "Nodes can be negative"

Let's visualize first 📌

🌳 Example 1 (All positive values)



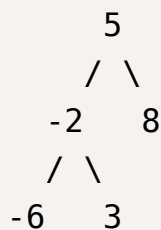
➡ Here, **every time you add**, your sum grows bigger!

You can safely say:

- Max sum path = go through all biggest branches (like greedy).

✅ Easy, no problem.

🌳 Example 2 (Some negative values)



Now imagine you're summing along a path.

Let's check some:

- $5 \rightarrow -2 \rightarrow -6 = -3$ ❌
- $5 \rightarrow -2 \rightarrow 3 = 6$ ✅
- $5 \rightarrow 8 = 13$ ✅✅

💡 Notice something?

Negative nodes can **drag down your total** sum.

So blindly adding "everything" isn't safe anymore.

🧠 Why this matters

When you write recursive code for sum-related problems, you need to **decide carefully**:

- Should you **include** that negative subtree?
- Or should you **ignore** it because it decreases the total?

That's why your teacher said:

| "Be sure to check whether nodes can be negative."

Because if they can, your logic must handle that possibility.

🧩 Example 1 — Simple total sum (all nodes)

If the problem says "find total sum of all nodes" — you can just sum everything, no worry.

```
def total_sum(node):  
    if not node:  
        return 0  
    return node.val + total_sum(node.left) + total_sum(n
```

```
ode.right)
```

- Just the same way as you `count_nodes` or `calculate height of tree`

✓ Works for all cases — no decision-making.

🧩 Example 2 — Maximum Path Sum (⚡ tricky one)

Now suppose we need to find **the path with the largest sum**, where the path can start and end *anywhere* in the tree.

That's where **negative nodes** can break things 😬

🧠 Core idea

Each node says:

"I can connect my left and right children —
but only if they give me **positive energy** ⚡!"

So if a child gives a *negative sum*,
that's *bad energy* 😬 — we **ignore it** (treat as 0).

```
def max_path_sum(root):
    # 📖 The "Champion's Notebook"
    max_sum = float('-inf')

    def dfs(node):
        if not node: return 0

        # Get "Job 2" reports from child, ignore 🚫 nega
        tives
        left = max(dfs(node.left), 0)
        right = max(dfs(node.right), 0)
```

```
# 🏆 Perform "Job 1" (Peak Check)
# This is the FIX we just proved!
max_sum = max(max_sum,
               node.val + left + right)
```

```
# 📈 Perform "Job 2" (Report Up)
# This is the logic we tested
return node.val + max(left, right)
```

```
dfs(root)
return max_sum
```

https://truongdinhtri.github.io/Interview-Prep/Coding%20Prepare/Week%203_Tree-Trie-Graph-Stack-Queue/Tree%20-%20Deployed/Summation_of_node.html

